DESENVOLVIMENTO WEB COM .NET

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Relatório 8 – Configuração por ambientes, segredos, contêineres e pipeline reprodutível

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: 2026

Instituto de Ciências Exatas e Tecnologia	**Disciplina:** Desenvolvimento Web com .NET **Título da Aula:** Configuração por ambientes, segredos, contêineres e pipeline reprodutível	**Relatório 8**

Unidade 8 – Build, implantação e CI/CD full-stack

Objetivo: Empacotar API e front-end, separar configuração de código e automatizar build, testes e publicação em um pipeline verificável.

1. Identificação e ambiente

Repositório: https://github.com/maiaralopes-dev/projetos (branch `ccr-836bb896-v5ejij`, pasta `PortalTarefas/`); commits `3f0201f`, `87ccb46`, `ac17bf4` e `6c526ad`. Docker 29.9.2 com BuildKit, Docker Compose 5.6.1. Imagens base com **tag exata + digest**: `node:22.23.4-alpine`, `mcr.microsoft.com/dotnet/sdk:10.0.403`, `mcr.microsoft.com/dotnet/aspnet:10.0.14`, `caddy:2.10.2-alpine` (proxy TLS) e Syft 1.33 (SBOM). Workflow `.github/workflows/portal-tarefas.yml` (GitHub Actions) e seu espelho executável `scripts/ci-local.sh`.

2. Resumo teórico aplicado ao projeto

Build, publish, imagem, contêiner, implantação e execução. *Build* compila e verifica o código; *publish* gera a pasta pronta para rodar (DLLs, dependências, `wwwroot`); a *imagem* é um pacote imutável e versionado com sistema de arquivos e comando de início; o *contêiner* é uma instância em execução da imagem; *implantar* é colocar uma versão específica da imagem em um ambiente com sua configuração; *executar* é o processo atendendo requisições.

Configuração por ambiente e precedência. No ASP.NET Core as fontes são lidas em ordem e a última vence: `appsettings.json` → `appsettings.{Ambiente}.json` → User Secrets (só em Development) → variáveis de ambiente → argumentos de linha de comando. Assim a **mesma imagem** roda em Staging e Production mudando só `ASPNETCORE_ENVIRONMENT` e variáveis (ex.: `ConnectionStrings__PortalTarefas`). Configurações obrigatórias são validadas na inicialização (`ValidateDataAnnotations().ValidateOnStart()`), e a mensagem cita a **chave** ausente, nunca o valor.

Segredos fora da imagem, do repositório e dos logs. Imagens e repositórios são copiados, armazenados em cache e lidos por muitas pessoas; um segredo gravado em uma camada permanece no histórico mesmo se “apagado” depois, e um segredo em log se espalha para ferramentas de observabilidade. Aqui: `.env` e User Secrets locais (no `.gitignore` e no `.dockerignore`), *secrets* do GitHub/ambientes no CI e, durante o build, a CA do proxy deste ambiente entra só como *BuildKit secret* — montada no `RUN` e ausente da imagem final.

Build multi-stage. O estágio `node` gera o bundle do React; o estágio `sdk` restaura e publica a API; o estágio final usa apenas o runtime `aspnet` e copia os artefatos. Ferramentas de build (Node, npm, SDK, código-fonte, testes) não chegam à imagem final, reduzindo

tamanho e superfície de ataque; a camada de `restore` fica em cache enquanto o `.csproj` não muda.

CI e CD. *Integração contínua* compila e testa cada alteração (PR e `main`) e só gera artefatos se tudo passar (*gate*). *Entrega contínua* promove o **mesmo** artefato por ambientes (staging → produção com aprovação). Rastreabilidade: a imagem recebe tag com o commit e rótulos OCI (`revision`, `version`), e `GET /api/versao` informa o commit em execução. *Rollback* = reimplantar a tag anterior, sem rebuild.

3. Código-fonte e evidências

Tabela 1 – Implantação integrada × separada

Critério	Integrada: API serve a SPA em `/app` (escolhida)	Separada: SPA em CDN/serviço próprio
Origem e autenticação	Mesma origem: cookie `SameSite=Strict`, sem CORS em produção	Origem distinta: CORS com credenciais ou tokens no cliente
Implantação e versão	Uma imagem = API + SPA sempre compatíveis	Dois artefatos; risco de versões descasadas do contrato
Escala e cache	Estáticos servidos pelo Kestrel (suficiente aqui)	CDN escala estáticos à parte; melhor para muito tráfego
Pipeline	Um build multi-stage	Pipelines independentes, deploys mais frequentes do front

Decisão: integrada, porque a segurança da unidade 6 (cookie `HttpOnly` sem tokens no JavaScript) depende da mesma origem e o volume de acesso não justifica CDN. Em produção, o TLS termina num proxy/ingress; no compose isso é simulado pelo **Caddy** (`https://localhost:8443 → api:8080`) e a API confia no `X-Forwarded-Proto` (`ASPNETCORE_FORWARDEDHEADERS_ENABLED=true`).

Dockerfile: 3 estágios; usuário não root (`USER $APP_UID`, UID 1654); arquivos da aplicação pertencentes ao root (somente leitura para o processo); única pasta gravável `/chaves` (chaves do Data Protection em volume); `HEALTHCHECK` que executa `dotnet PortalTarefas.Web.dll --sondar-saude` (consulta `/health/live` sem instalar curl). No compose: `read_only: true`, `tmpfs /tmp`, `cap_drop: [ALL]` e `no-new-privileges`. Imagem final: **129 MB comprimidos (462 MB em disco)**, sem Node, npm, curl, wget nem SDK; a busca pelas senhas locais em todo o sistema de arquivos da imagem retornou 0 ocorrências.

Problemas reais encontrados ao executar o contêiner e corrigidos: (1) o volume de chaves pertencia ao root e o app (UID 1654) não conseguia gravar → pasta criada na imagem com dono `app`; (2) cookies `Secure` e antiforgery recusavam HTTP puro → TLS terminado no proxy reverso; (3) o build limpo acusou o aviso ASP0026 (`[AllowAnonymous]` da classe anulava o `[Authorize]` do logout) → atributos movidos para as ações públicas.

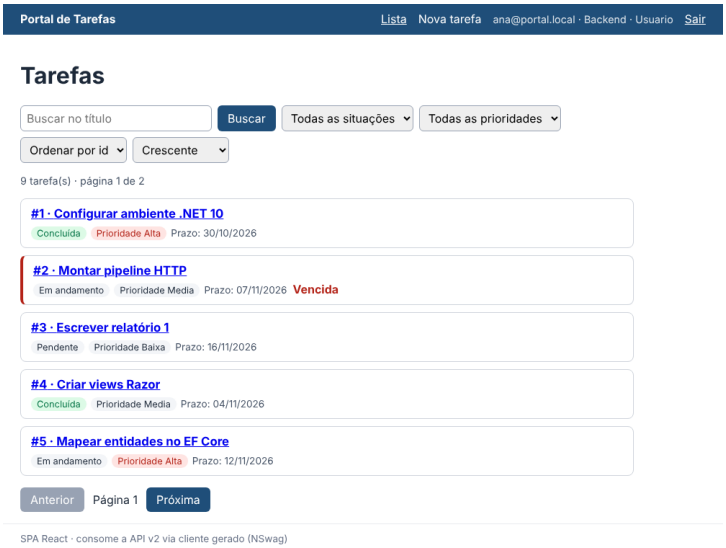


Figura 1 – SPA servida pelo contêiner da API atrás do proxy TLS, após login

4. Testes e validações obrigatórias

Tabela 2 – Testes obrigatórios

Caso exigido	Resultado observado	Evidência
Build limpo a partir de clone novo	`dotnet build -c Release` 0 avisos; 70 testes .NET e 22 do front aprovados; `npm ci` + build ok; árvore Git limpa após o build	`02-build-clone-limpo.txt`
Contêiner com configuração externa, sem segredo na imagem	Conexão, senha de seed e URL vindas de variáveis/`.env`; `ENV` e histórico da imagem sem segredos; 0 ocorrências das senhas na imagem	`03-inspecao-imagem.txt`, `04-compose-execucao.txt`
Health Check do serviço	Docker: `healthy`; `/health/live` e `/health/ready` 200 via proxy TLS; mesma imagem em Production também `Healthy`	`04…`, `07-config-ambientes-e-validacao.txt`
Falha de teste bloqueia a publicação	`CI_FALHA_DELIBERADA=1`: teste `Gate_de_publicacao` falha na etapa 3/8, código de saída 1, nenhum artefato publicado	`06-ci-local-falha-bloqueia.txt`
Pipeline para alteração válida + rastreabilidade	8/8 etapas; imagem `portal-tarefas:ac17bf4`, smoke test 200, SBOM e `manifesto.json` com commit e id da imagem	`05-ci-local-sucesso.txt`

Observação de transparência: o workflow do GitHub Actions está versionado, mas só dispara em *pull request* ou *push* na `main` (e exige o secret `CI_SQL_SA_PASSWORD`); por isso as execuções registradas foram feitas com `scripts/ci-local.sh`, que executa as mesmas etapas na mesma ordem. Ao abrir o PR, o workflow roda no GitHub.

5. Aprimoramentos

- **SBOM** (SPDX, Syft): 160 pacotes inventariados (97 do Debian, 56 NuGet...), publicado como artefato.
- **Análise de vulnerabilidades com política de bloqueio:** `dotnet list package --vulnerable --include-transitive` (falha em High/Critical), `npm audit --audit-level=high`

e Trivy na imagem (falha em Critical com correção disponível). Resultado local: nenhuma vulnerabilidade; antes, a análise apontou `System.Security.Cryptography.Pkcs 9.0.11` (High), corrigido fixando a 10.0.14.

- **Staging com aprovação antes de produção:** jobs `staging` e `producao` com *environments* do GitHub (aprovação manual em produção) que promovem a **mesma** tag de imagem; rollback documentado.
- **Publicação sem credencial longa:** push ao GHCR com o `GITHUB_TOKEN` efêmero do job, com permissões mínimas (`contents: read`, `packages: write` só onde necessário).

Referências

MICROSOFT. **Configuration in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/configuration/. Acesso em: 9 nov. 2026.

MICROSOFT. **Safe storage of app secrets in development in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/security/app-secrets. Acesso em: 9 nov. 2026.

MICROSOFT. **Options pattern in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/configuration/options. Acesso em: 9 nov. 2026.

MICROSOFT. **Containerize a .NET app**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/dotnet/core/docker/build-container. Acesso em: 9 nov. 2026.

MICROSOFT. **Configure ASP.NET Core to work with proxy servers and load balancers**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/host-and-deploy/proxy-load-balancer. Acesso em: 9 nov. 2026.

DOCKER. **Multi-stage builds** e **Build secrets**. Docker Docs, 2025. Disponível em: https://docs.docker.com/build/building/multi-stage/. Acesso em: 9 nov. 2026.

GITHUB. **Using environments for deployment** e **Publishing Docker images**. GitHub Docs, 2025. Disponível em: https://docs.github.com/actions. Acesso em: 9 nov. 2026.

OPEN CONTAINER INITIATIVE. **Image Format Specification: Annotations**. 2024. Disponível em: https://github.com/opencontainers/image-spec/blob/main/annotations.md. Acesso em: 9 nov. 2026.

SPDX. **SPDX Specification 2.3**. Linux Foundation, 2022. Disponível em: https://spdx.github.io/spdx-spec/v2.3/. Acesso em: 9 nov. 2026.

Apêndice A – Arquivos de build, configuração e pipeline

Arquivo: `Dockerfile` (commit `6c526ad`)

```
# syntax=docker/dockerfile:1
# Imagem única (implantação integrada): API ASP.NET Core + SPA React servida em /app.
# Imagens base com tag exata + digest: o build não muda se a tag for republicada.

# ---------- 1. Front-end: instala dependências travadas e gera o bundle ----------
FROM node:22.23.4-alpine@sha256:0e1754758c2df4cfa3e68d59e2e1b7374bb7fd9ea6328eaaacf15cc934af8202 AS frontend
WORKDIR /src
COPY src/PortalTarefas.Frontend/package.json src/PortalTarefas.Frontend/package-lock.json ./
# CA extra opcional (proxy corporativo) via BuildKit secret: usada só durante o RUN e nunca gravada na imagem.
RUN --mount=type=secret,id=ca_extra,required=false \
    if [ -f /run/secrets/ca_extra ]; then export NODE_EXTRA_CA_CERTS=/run/secrets/ca_extra; fi; \
    npm ci --ignore-scripts --no-audit --no-fund
COPY src/PortalTarefas.Frontend/ ./
RUN npm run build                                  # tsc -b && vite build -> /src/dist (base /app/)

# ---------- 2. Back-end: restore (camada em cache) -> publish ----------
FROM
mcr.microsoft.com/dotnet/sdk:10.0.403@sha256:212fe4cb74627d5adf9817a4088906ae3edff1bd853bb27565d4b46731efe61e AS
backend
WORKDIR /src
COPY global.json ./
COPY src/PortalTarefas.Web/PortalTarefas.Web.csproj src/PortalTarefas.Web/
RUN --mount=type=secret,id=ca_extra,required=false \
    if [ -f /run/secrets/ca_extra ]; then export SSL_CERT_FILE=/run/secrets/ca_extra; fi; \
    dotnet restore src/PortalTarefas.Web/PortalTarefas.Web.csproj
COPY src/PortalTarefas.Web/ src/PortalTarefas.Web/
COPY docs/openapi/ docs/openapi/
RUN dotnet publish src/PortalTarefas.Web/PortalTarefas.Web.csproj -c Release -o /app/publish \
      --no-restore -p:OpenApiGenerateDocuments=false -p:UseAppHost=false
COPY --from=frontend /src/dist /app/publish/wwwroot/app

# ---------- 3. Execução: só o runtime ASP.NET Core + arquivos publicados ----------
FROM
mcr.microsoft.com/dotnet/aspnet:10.0.14@sha256:1a48394b6f29d3a32c79d9edecd4abda0ecbe8776d567dbfc4590ed6979c740c
AS final
ARG APP_VERSION=dev
ARG APP_COMMIT=local
LABEL org.opencontainers.image.title="portal-tarefas" \
      org.opencontainers.image.version="${APP_VERSION}" \
      org.opencontainers.image.revision="${APP_COMMIT}" \
      org.opencontainers.image.source="https://github.com/maiaralopes-dev/projetos"
ENV ASPNETCORE_HTTP_PORTS=8080 \
    ASPNETCORE_ENVIRONMENT=Production \
    ASPNETCORE_FORWARDEDHEADERS_ENABLED=true \
    DOTNET_NOLOGO=1 \
    APP_VERSION=${APP_VERSION} \
    APP_COMMIT=${APP_COMMIT}
WORKDIR /app
COPY --from=backend --chown=root:root /app/publish ./
# Única pasta gravável pelo app: chaves do Data Protection (montada como volume no compose).
RUN mkdir -p /chaves && chown $APP_UID:$APP_UID /chaves && chmod 700 /chaves
# Usuário não privilegiado já existente na imagem oficial (UID 1654). Arquivos da aplicação pertencem ao root
(somente leitura para o app).
USER $APP_UID
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=5s --start-period=30s --retries=3 \
  CMD ["dotnet", "PortalTarefas.Web.dll", "--sondar-saude"]
ENTRYPOINT ["dotnet", "PortalTarefas.Web.dll"]
```

Arquivo: `.dockerignore` (commit `6c526ad`)

```
# Nada de segredos, artefatos locais ou dependências instaladas no contexto de build.
**/bin/
**/obj/
**/node_modules/
**/dist/
**/.env
**/.env.*
!**/.env.example
!src/PortalTarefas.Frontend/.env.development
**/*.user
**/secrets.json
.git/
.vs/
docs/evidencias/
docs/relatorios/
tests/
TestResults/
artifacts/
```

Arquivo: `compose.yaml` (commit `6c526ad`)

```
# Ambiente local completo (simula "teste/staging"): banco, dependência externa e a imagem da aplicação.
# Segredos vêm do arquivo .env (não versionado) — ver .env.example. Nada sensível é gravado na imagem.
name: portal-tarefas
```

```
services:
  sql:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: ${MSSQL_SA_PASSWORD:?defina MSSQL_SA_PASSWORD no .env}
    healthcheck:
      test: ["CMD-SHELL", "/opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P \"$$MSSQL_SA_PASSWORD\" -Q
'SELECT 1' -b -o /dev/null"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 20s

  calendario:
    image: node:22.23.4-alpine@sha256:0e1754758c2df4cfa3e68d59e2e1b7374bb7fd9ea6328eaaacf15cc934af8202
    command: ["node", "/srv/dependencia-calendario.js"]
    volumes:
      - ./scripts/dependencia-calendario.js:/srv/dependencia-calendario.js:ro

  api:
    image: portal-tarefas:${APP_VERSION:-dev}
    build:
      context: .
      args:
        APP_VERSION: ${APP_VERSION:-dev}
        APP_COMMIT: ${APP_COMMIT:-local}
    depends_on:
      sql: { condition: service_healthy }
      calendario: { condition: service_started }
    expose: ["8080"]                    # sem porta publicada: o acesso externo passa pelo proxy TLS
    environment:
      ASPNETCORE_ENVIRONMENT: Staging
      ConnectionStrings__PortalTarefas: "Server=sql,1433;Database=PortalTarefas;User Id=sa;Password=$
{MSSQL_SA_PASSWORD};TrustServerCertificate=True;Encrypt=True"
      SeedUsuarios__SenhaInicial: ${SEED_SENHA:-}
      Banco__AplicarMigrations: "true"
      Dependencias__Calendario__Url: http://calendario:5099/
      DataProtection__Caminho: /chaves
    read_only: true                     # sistema de arquivos somente leitura
    tmpfs: [/tmp]
    volumes:
      - chaves-dp:/chaves               # chaves do Data Protection sobrevivem à recriação do contêiner
    security_opt: ["no-new-privileges:true"]
    cap_drop: [ALL]

  # Proxy reverso que termina o TLS (como um ingress/load balancer faria em produção).
  # A API recebe X-Forwarded-Proto=https (ASPNETCORE_FORWARDEDHEADERS_ENABLED) e emite cookies Secure.
  proxy:
    image: caddy:2.10.2-alpine@sha256:4c6e91c6ed0e2fa03efd5b44747b625fec79bc9cd06ac5235a779726618e530d
    command: ["caddy", "reverse-proxy", "--from", "https://localhost:8443", "--to", "api:8080", "--internal-
certs"]
    ports:
      - "8443:8443"
    depends_on:
      api: { condition: service_healthy }

volumes:
  chaves-dp:
```

Arquivo: `.env.example` (commit `6c526ad`)

```
# ================================================================================
# Variáveis de configuração do Portal de Tarefas — SEM valores secretos.
# Copie para .env (ignorado pelo Git) para uso local com scripts/ e compose.yaml.
# Em produção, forneça os valores pelo secret store do provedor.
# ================================================================================

# --- Usadas pelo compose.yaml / scripts locais ---
MSSQL_SA_PASSWORD=          # senha do 'sa' do SQL Server local (somente desenvolvimento)
SEED_SENHA=                 # senha inicial dos usuários de teste (ana, bruno, carla, visitante)
APP_VERSION=                # versão gravada na imagem (ex.: 1.0.0+<sha>)
APP_COMMIT=                 # commit que gerou a imagem

# --- Lidas pela aplicação (formato variável de ambiente: seção__chave) ---
# ConnectionStrings__PortalTarefas      OBRIGATÓRIA. Conexão SQL Server. Dev: User Secrets.
# Dependencias__Calendario__Url         OBRIGATÓRIA. URL do serviço de calendário (validada no início).
# SeedUsuarios__SenhaInicial            Opcional. Sem ela, usuários de teste não são criados.
# Banco__AplicarMigrations              true/false. Aplica migrations + seed ao iniciar (dev/teste).
# DataProtection__Caminho               Pasta persistente das chaves que cifram o cookie de sessão.
# Cors__Origens__0                      Origem autorizada para CORS (dev: http://localhost:5173).
# RateLimit__Api__Limite / __JanelaSegundos, RateLimit__Publica__..., RateLimit__Login__Limite
# Seguranca__IntervaloValidacaoStamp    Intervalo de verificação do security stamp (padrão 00:00:00).
# OTEL_EXPORTER_OTLP_ENDPOINT           Coletor OTLP (ex.: http://localhost:4317). Vazio = sem exportação.
# Telemetria__Console                   true para exportar traces/métricas no console.
# APP_VERSION / APP_COMMIT              Gravadas na imagem; expostas em GET /api/versao.
# ASPNETCORE_ENVIRONMENT                Development | Staging | Production.
```

Arquivo: `scripts/ci-local.sh` (commit `6c526ad`)

```
#!/usr/bin/env bash
# Executa localmente as MESMAS etapas do workflow .github/workflows/portal-tarefas.yml, na mesma ordem.
# Qualquer falha encerra o script (set -e) ANTES da etapa de publicação.
set -euo pipefail
cd "$(dirname "$0")/.."
SHA=$(git rev-parse --short HEAD)
etapa() { echo; echo "::: [$(date +%H:%M:%S)] $*"; }
source .env 2>/dev/null || true
export PORTAL_SQL_TESTES="Server=localhost,1433;User Id=sa;Password=$
{MSSQL_SA_PASSWORD};TrustServerCertificate=True"

etapa "1/8 dotnet restore";  dotnet restore --nologo -v q
etapa "2/8 dotnet build (Release)"; dotnet build -c Release --no-restore --nologo -v q -
p:OpenApiGenerateDocuments=false 2>&1 | grep -E "error|Warn|Build succeeded" || true
etapa "3/8 dotnet test"; dotnet test -c Release --no-build --nologo --logger "trx;LogFileName=testes.trx" --
results-directory artifacts/testes 2>&1 | grep -E "Passed!|Failed!|Failed |error"
etapa "4/8 npm ci";   (cd src/PortalTarefas.Frontend && npm ci --no-audit --no-fund --silent)
etapa "5/8 npm test"; (cd src/PortalTarefas.Frontend && npx vitest run 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -E
"Tests |FAIL")
etapa "6/8 npm run build"; (cd src/PortalTarefas.Frontend && npm run build 2>&1 | grep -E "built|error")
etapa "7/8 imagem + smoke test"
scripts/docker-build.sh > artifacts/docker-build.log 2>&1 && echo "imagem portal-tarefas:$SHA construída"
docker rm -f smoke >/dev/null 2>&1 || true
docker run -d --name smoke -p 18080:8080 -e ConnectionStrings__PortalTarefas="Server=nao-usado" \
  -e Dependencias__Calendario__Url=http://nao-usado/ portal-tarefas:$SHA >/dev/null
for i in $(seq 1 30); do curl -sf http://localhost:18080/health/live >/dev/null && break; sleep 1; done
echo "smoke /health/live: $(curl -s -o /dev/null -w '%{http_code}' http://localhost:18080/health/live)"
echo "smoke /api/versao:  $(curl -s http://localhost:18080/api/versao)"
docker rm -f smoke >/dev/null
etapa "8/8 publicação de artefatos (só chega aqui se tudo acima passou)"
DEST=artifacts/publicado/$SHA; mkdir -p $DEST
docker save portal-tarefas:$SHA | gzip > $DEST/imagem.tar.gz
cp -r src/PortalTarefas.Frontend/dist $DEST/frontend-dist
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock anchore/syft:v1.33.0 portal-tarefas:$SHA -o spdx-
json > $DEST/sbom.spdx.json 2>/dev/null
cat > $DEST/manifesto.json <<JSON
{ "commit": "$(git rev-parse HEAD)", "imagem": "portal-tarefas:$SHA",
  "imagemId": "$(docker image inspect --format '{{.Id}}' portal-tarefas:$SHA)",
  "geradoEm": "$(date -u +%FT%TZ)", "testes": "artifacts/testes/testes.trx" }
JSON
ls -la $DEST | awk '{print $5, $9}'
echo "PUBLICADO: $DEST"
```

Arquivo: `scripts/docker-build.sh` (commit `6c526ad`)

```
#!/usr/bin/env bash
# Constrói a imagem com tag ligada ao commit. Se houver proxy/CA corporativa (como neste ambiente),
# repassa-os ao BuildKit sem gravá-los na imagem (args de proxy predefinidos não entram no histórico).
set -euo pipefail
SHA=$(git rev-parse --short HEAD)
VERSAO=${APP_VERSION:-1.0.0+$SHA}
EXTRA=()
if [ -n "${HTTPS_PROXY:-}" ]; then EXTRA+=(--network host --build-arg HTTPS_PROXY --build-arg HTTP_PROXY --build-
arg NO_PROXY); fi
if [ -f /root/.ccr/ca-bundle.crt ]; then EXTRA+=(--secret id=ca_extra,src=/root/.ccr/ca-bundle.crt); fi
echo "\$ docker build --build-arg APP_VERSION=$VERSAO --build-arg APP_COMMIT=$SHA -t portal-tarefas:$SHA -t
portal-tarefas:dev ."
DOCKER_BUILDKIT=1 docker build --progress=plain "${EXTRA[@]}" \
  --build-arg APP_VERSION="$VERSAO" --build-arg APP_COMMIT="$SHA" \
  -t "portal-tarefas:$SHA" -t portal-tarefas:dev .
```

Arquivo: `src/PortalTarefas.Web/appsettings.Production.json` (commit `6c526ad`)

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Warning"
    }
  },
  "Banco": { "AplicarMigrations": false },
  "AllowedHosts": "*"
}
```

Arquivo: `src/PortalTarefas.Web/Configuracao/Opcoes.cs` (commit `6c526ad`)

```
using System.ComponentModel.DataAnnotations;

namespace PortalTarefas.Web.Configuracao;

/// <summary>Configuração obrigatória validada na inicialização (ValidateOnStart).</summary>
public sealed class DependenciasOptions
{
    public const string Secao = "Dependencias:Calendario";
```

```
    [Required(ErrorMessage = "Dependencias:Calendario:Url é obrigatória."), Url(ErrorMessage =
"Dependencias:Calendario:Url deve ser uma URL absoluta.")]
    public string Url { get; set; } = "";
}

public sealed class ConexaoOptions
{
    [Required(ErrorMessage = "ConnectionStrings:PortalTarefas é obrigatória (use User Secrets em dev ou a
variável ConnectionStrings__PortalTarefas).")]
    public string? PortalTarefas { get; set; }
}

/// <summary>Identificação da versão implantada (rastreabilidade commit → artefato → execução).</summary>
public sealed record VersaoDto(string Versao, string Commit, string Ambiente);
```

Arquivo: `src/PortalTarefas.Web/Infraestrutura/SondaSaude.cs` (commit `6c526ad`)

```
namespace PortalTarefas.Web.Infraestrutura;

/// <summary>
/// Modo "sonda": `dotnet PortalTarefas.Web.dll --sondar-saude` consulta /health/live do próprio contêiner
/// e encerra com 0 (saudável) ou 1. Evita instalar curl/wget na imagem final apenas para o HEALTHCHECK.
/// </summary>
public static class SondaSaude
{
    public static async Task<int> ExecutarAsync()
    {
        var porta = Environment.GetEnvironmentVariable("ASPNETCORE_HTTP_PORTS")?.Split(';')[0] ?? "8080";
        try
        {
            using var http = new HttpClient { Timeout = TimeSpan.FromSeconds(3) };
            using var resp = await http.GetAsync($"http://127.0.0.1:{porta}/health/live");
            return resp.IsSuccessStatusCode ? 0 : 1;
        }
        catch
        {
            return 1;
        }
    }
}
```

Arquivo: `.github/workflows/portal-tarefas.yml` (commit `6c526ad`)

```
# CI/CD do Portal de Tarefas (pasta PortalTarefas/). Etapas espelhadas em PortalTarefas/scripts/ci-local.sh.
name: portal-tarefas

on:
  pull_request:
    paths: ["PortalTarefas/**", ".github/workflows/portal-tarefas.yml"]
  push:
    branches: [main]
    paths: ["PortalTarefas/**", ".github/workflows/portal-tarefas.yml"]
  workflow_dispatch:

permissions:
  contents: read            # mínimo por padrão; jobs que publicam pedem mais

concurrency:
  group: portal-tarefas-${{ github.ref }}
  cancel-in-progress: true

defaults:
  run:
    working-directory: PortalTarefas

jobs:
  backend:
    runs-on: ubuntu-24.04
    services:
      sql:                                  # banco descartável só para os testes de integração
        image: mcr.microsoft.com/mssql/server:2022-latest
        env:
          ACCEPT_EULA: "Y"
          MSSQL_SA_PASSWORD: ${{ secrets.CI_SQL_SA_PASSWORD }}
        ports: ["1433:1433"]
        options: >-
          --health-cmd "/opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P \"$MSSQL_SA_PASSWORD\" -Q 'SELECT
1' -b -o /dev/null"
          --health-interval 10s --health-timeout 5s --health-retries 12
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          global-json-file: PortalTarefas/global.json   # versão de SDK explícita e reproduzível
      - run: dotnet restore
      - run: dotnet build -c Release --no-restore -p:OpenApiGenerateDocuments=false
      - name: dotnet test
        env:
          PORTAL_SQL_TESTES: "Server=localhost,1433;User Id=sa;Password=$
{{ secrets.CI_SQL_SA_PASSWORD }};TrustServerCertificate=True"
```

```
        run: dotnet test -c Release --no-build --logger "trx;LogFileName=testes.trx" --results-directory
TestResults
      - name: Dependências .NET vulneráveis (bloqueia em High/Critical)
        run: |
          dotnet list package --vulnerable --include-transitive | tee vulneraveis.txt
          ! grep -qE "\b(High|Critical)\b" vulneraveis.txt
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: testes-dotnet, path: PortalTarefas/TestResults }

  frontend:
    runs-on: ubuntu-24.04
    defaults:
      run:
        working-directory: PortalTarefas/src/PortalTarefas.Frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22.23.4"
          cache: npm
          cache-dependency-path: PortalTarefas/src/PortalTarefas.Frontend/package-lock.json
      - run: npm ci
      - run: npm audit --audit-level=high --omit=dev
      - run: npx vitest run
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with: { name: frontend-dist, path: PortalTarefas/src/PortalTarefas.Frontend/dist }

  imagem:
    needs: [backend, frontend]               # só roda se TODOS os testes passaram
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      packages: write                        # publicar no GHCR com o GITHUB_TOKEN efêmero (sem credencial longa)
    outputs:
      tag: ${{ steps.meta.outputs.tag }}
    steps:
      - uses: actions/checkout@v4
      - id: meta
        run: echo "tag=ghcr.io/${GITHUB_REPOSITORY,,}/portal-tarefas:${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"
      - name: Build da imagem (tag = commit)
        run: |
          docker build --build-arg APP_VERSION="1.0.${{ github.run_number }}+${GITHUB_SHA::7}" \
                       --build-arg APP_COMMIT="${GITHUB_SHA}" -t "${{ steps.meta.outputs.tag }}" .
      - name: Smoke test do contêiner
        run: |
          docker run -d --name smoke -p 8080:8080 -e ConnectionStrings__PortalTarefas=Server=nao-usado \
            -e Dependencias__Calendario__Url=http://nao-usado/ "${{ steps.meta.outputs.tag }}"
          for i in $(seq 1 30); do curl -sf localhost:8080/health/live && break; sleep 1; done
          curl -sf localhost:8080/api/versao
      - name: SBOM (SPDX)
        uses: anchore/sbom-action@v0
        with:
          image: ${{ steps.meta.outputs.tag }}
          format: spdx-json
          output-file: sbom.spdx.json
      - name: Varredura de vulnerabilidades da imagem (falha em Critical corrigível)
        uses: aquasecurity/trivy-action@0.33.1
        with:
          image-ref: ${{ steps.meta.outputs.tag }}
          severity: CRITICAL
          ignore-unfixed: true
          exit-code: "1"
      - uses: actions/upload-artifact@v4
        with: { name: sbom, path: PortalTarefas/sbom.spdx.json }
      - name: Publicar no GHCR (somente main)
        if: github.ref == 'refs/heads/main'
        run: |
          echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u "${{ github.actor }}" --password-stdin
          docker push "${{ steps.meta.outputs.tag }}"

  staging:
    if: github.ref == 'refs/heads/main'
    needs: imagem
    runs-on: ubuntu-24.04
    environment: staging                      # segredos do ambiente + regras de proteção no GitHub
    steps:
      - uses: actions/checkout@v4
      - name: Implantar em ambiente de teste efêmero (compose) e verificar
        env:
          MSSQL_SA_PASSWORD: ${{ secrets.STAGING_SQL_SA_PASSWORD }}
          SEED_SENHA: ${{ secrets.STAGING_SEED_SENHA }}
        run: |
          docker pull "${{ needs.imagem.outputs.tag }}"
          docker tag "${{ needs.imagem.outputs.tag }}" portal-tarefas:staging
          APP_VERSION=staging docker compose up -d --no-build
          for i in $(seq 1 60); do curl -sf localhost:8080/health/ready && break; sleep 2; done
          curl -sf localhost:8080/health/ready && curl -sf localhost:8080/api/versao
          echo "versão implantada: ${{ needs.imagem.outputs.tag }}" >> "$GITHUB_STEP_SUMMARY"

  producao:
```

```
    needs: [imagem, staging]
    runs-on: ubuntu-24.04
    environment: production                  # exige aprovação manual (required reviewers)
    steps:
      - name: Promover a MESMA imagem aprovada em staging
        run: |
          echo "Promover ${{ needs.imagem.outputs.tag }} (sem rebuild)."
          echo "Rollback: reimplantar a tag anterior registrada no resumo do run anterior."
```

Apêndice B – Evidências

Pipeline local: alteração válida (8/8 etapas, publicação e manifesto)

```
$ scripts/ci-local.sh   # alteração válida

::: [02:07:22] 1/8 dotnet restore

::: [02:07:23] 2/8 dotnet build (Release)
Build succeeded.
    0 Warning(s)

::: [02:07:26] 3/8 dotnet test
Passed!  - Failed:     0, Passed:    70, Skipped:     0, Total:    70, Duration: 8 s - PortalTarefas.Tests.dll
(net10.0)

::: [02:07:36] 4/8 npm ci

::: [02:07:39] 5/8 npm test
      Tests  22 passed (22)

::: [02:07:40] 6/8 npm run build
✓ built in 138ms

::: [02:07:41] 7/8 imagem + smoke test
imagem portal-tarefas:ac17bf4 construída
smoke /health/live: 200
smoke /api/versao:  {"versao":"ac17bf4","commit":"ac17bf4","ambiente":"Production"}

::: [02:07:44] 8/8 publicação de artefatos (só chega aqui se tudo acima passou)

4096 .
4096 ..
4096 frontend-dist
127674623 imagem.tar.gz
261 manifesto.json
3232281 sbom.spdx.json
PUBLICADO: artifacts/publicado/ac17bf4
código de saída: 0
{ "commit": "ac17bf430d348c21982cc1d014c64a1c50013c7d", "imagem": "portal-tarefas:ac17bf4",
  "imagemId": "sha256:79586aeebfb04b3247d8f4e23431d4bfc5ee6b98e8de8614a2dfcf77cca5bed3",
  "geradoEm": "2026-11-09T02:07:56Z", "testes": "artifacts/testes/testes.trx" }
```

Pipeline local: falha deliberada bloqueia a publicação

```
$ CI_FALHA_DELIBERADA=1 scripts/ci-local.sh   # teste falha de propósito

::: [02:07:56] 1/8 dotnet restore

::: [02:07:57] 2/8 dotnet build (Release)
Build succeeded.
    0 Warning(s)

::: [02:07:58] 3/8 dotnet test
  Failed PortalTarefas.Tests.Atividade8PipelineTests.Gate_de_publicacao [1 ms]
Failed!  - Failed:     1, Passed:    69, Skipped:     0, Total:    70, Duration: 6 s - PortalTarefas.Tests.dll
(net10.0)
código de saída: 1

$ ls artifacts/publicado   # nada foi publicado
ls: cannot access 'artifacts/publicado': No such file or directory
```

Inspeção da imagem

```
===== Imagem portal-tarefas:ac17bf4 =====
tamanho em disco: 462MB

-- Rótulos (rastreabilidade):
org.opencontainers.image.revision=ac17bf4
org.opencontainers.image.source=https://github.com/maiaralopes-dev/projetos
org.opencontainers.image.title=portal-tarefas
org.opencontainers.image.version=ac17bf4
-- Usuário de execução: 1654
uid=1654(app) gid=1654(app) groups=1654(app)

-- Variáveis de ambiente gravadas na imagem:
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
APP_UID=1654
ASPNETCORE_HTTP_PORTS=8080
DOTNET_RUNNING_IN_CONTAINER=true
DOTNET_VERSION=10.0.14
ASPNET_VERSION=10.0.14
ASPNETCORE_ENVIRONMENT=Production
ASPNETCORE_FORWARDEDHEADERS_ENABLED=true
DOTNET_NOLOGO=1
APP_VERSION=ac17bf4
APP_COMMIT=ac17bf4

-- Histórico de camadas da imagem final:
0B    ENTRYPOINT ["dotnet" "PortalTarefas.Web.dll"]
0B    HEALTHCHECK {Test:[CMD dotnet PortalTarefas.…
0B    EXPOSE [8080/tcp]
0B    USER 1654
8.19kB    RUN |2 APP_VERSION=ac17bf4 APP_COMMIT=ac17bf…
89.1MB    COPY --chown=root:root /app/publish ./ # bui…
8.19kB    WORKDIR /app
0B    ENV ASPNETCORE_HTTP_PORTS=8080 ASPNETCORE_EN…
0B    LABEL org.opencontainers.image.title=portal-…
0B    ARG APP_COMMIT=ac17bf4
0B    ARG APP_VERSION=ac17bf4
27.4MB    COPY /dotnet /usr/share/dotnet # buildkit
0B    ENV ASPNET_VERSION=10.0.14

-- Arquivos de configuração presentes em /app:
/app/appsettings.Development.json
/app/appsettings.Production.json
/app/appsettings.Staging.json
/app/appsettings.json
nenhum .env/secrets.json em /app
ocorrências das senhas locais em todo o sistema de arquivos da imagem: 0

-- Ferramentas de build/rede ausentes na imagem final:
node ausente
npm ausente
curl ausente
wget ausente
SDKs .NET instalados: 0
```

Compose: contêineres, health check e requisições via proxy TLS

```
$ APP_VERSION=ac17bf4 docker compose up -d --no-build   # segredos vêm do .env (fora do Git)
 Container portal-tarefas-sql-1 Created
 Container portal-tarefas-calendario-1 Created
 Container portal-tarefas-api-1 Created
 Container portal-tarefas-proxy-1 Created
 Container portal-tarefas-calendario-1 Started
 Container portal-tarefas-sql-1 Started
 Container portal-tarefas-sql-1 Healthy
 Container portal-tarefas-api-1 Started
 Container portal-tarefas-api-1 Healthy
 Container portal-tarefas-proxy-1 Started

Estado do HEALTHCHECK do contêiner da API: healthy
SERVICE      IMAGE                                                                                STATUS
PORTS
api          portal-tarefas:ac17bf4                                                               Up 5 seconds
(healthy)    8080/tcp
calendario   node:22.23.4-alpine@sha256:0e1754758c2df4cfa3e68d59e2e1b7374bb7fd9ea6328eaaacf15cc934af8202   Up 11 seconds
proxy        caddy:2.10.2-alpine@sha256:4c6e91c6ed0e2fa03efd5b44747b625fec79bc9cd06ac5235a779726618e530d   Up Less than a
second     80/tcp, 443/tcp, 2019/tcp, 443/udp, 0
sql          mcr.microsoft.com/mssql/server:2022-latest                                           Up 11 seconds
(healthy)   1433/tcp

### Liveness via proxy TLS
$ curl https://localhost:8443/health/live
HTTP/2 200
cache-control: no-store, no-cache
content-type: text/plain
x-correlation-id: 59ebe2136a3746bcbcd06c48e92000f3
Healthy
### Readiness (banco + calendário)
$ curl https://localhost:8443/health/ready
HTTP/2 200
cache-control: no-store, no-cache
content-type: text/plain
x-correlation-id: aa21e7e2c10f45528fb1bb010540c3e1
Healthy
### Versão implantada (commit -> imagem -> execução)
$ curl https://localhost:8443/api/versao
HTTP/2 200
content-type: application/json; charset=utf-8
x-correlation-id: 4fe0cbc6153e4d9197811a8c91aa2e0b
{
    "versao": "ac17bf4",
    "commit": "ac17bf4",
    "ambiente": "Staging"
}

### SPA servida pela API (implantação integrada)
$ curl https://localhost:8443/app/tarefas
HTTP/2 200
content-type: text/html
```

```
x-correlation-id: 64c793712d0b4100a8d18b6de4d7808c
<!doctype html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
### Sessão anônima: cookies Secure emitidos atrás do proxy
$ curl https://localhost:8443/api/v2/auth/sessao
HTTP/2 200
cache-control: no-cache, no-store
content-type: application/json; charset=utf-8
set-cookie: portal.af=<valor omitido>; path=/; secure; samesite=strict; httponly
set-cookie: XSRF-TOKEN=<valor omitido>; path=/; secure; samesite=strict
x-correlation-id: 553c28f3c1f943c997bcfbcb81d8f130
{
    "autenticado": false,
    "usuario": null,
    "equipe": null,
### Rota sem endpoint (ex.: UI Scalar, só existe em Development)
$ curl https://localhost:8443/scalar/v2
HTTP/2 302
location: https://localhost:8443/Conta/Login?ReturnUrl=%2Fscalar%2Fv2
x-correlation-id: 869c5be8354d41f99bad4c1e9ff1a45b
-- Configuração vista pelo contêiner (nomes; valores ocultos):
ASPNETCORE_ENVIRONMENT=<definida>
Banco__AplicarMigrations=<definida>
ConnectionStrings__PortalTarefas=<definida>
DataProtection__Caminho=<definida>
```

Mesma imagem em Production e validação de configuração

```
===== Mesma imagem em Production (sem alterar código), na rede do compose =====
$ docker run -d --name prod-sim --network portal-tarefas_default -e ASPNETCORE_ENVIRONMENT=Production -e
ConnectionStrings__PortalTarefas=<omitida> -e Dependencias__Calen …
### Versão (Production)
$ curl http://localhost:8081/api/versao
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
X-Correlation-Id: daadcb33ab5b4dd4adf1658d1122fc5d
{
    "versao": "ac17bf4",
    "commit": "ac17bf4",
    "ambiente": "Production"
}

### Readiness (Production)
$ curl http://localhost:8081/health/ready
HTTP/1.1 200 OK
Content-Type: text/plain
Cache-Control: no-store, no-cache
X-Correlation-Id: ad99b4e3bdb94325b866efeced0a99dd
Healthy

===== Configuração obrigatória ausente: falha na inicialização, mensagem cita a chave e não o valor =====
$ docker run --rm -e Dependencias__Calendario__Url= -e
ConnectionStrings__PortalTarefas='Server=x;Password=NAO_DEVE_APARECER' portal-tarefas:ac17bf4
Unhandled exception. Microsoft.Extensions.Options.OptionsValidationException: DataAnnotation validation failed
for 'DependenciasOptions' members: 'Url' with the error: 'D …
código de saída do contêiner: 139
ocorrências de 'NAO_DEVE_APARECER' na saída: 0

$ docker run --rm portal-tarefas:ac17bf4   # sem connection string
Unhandled exception. System.InvalidOperationException: Connection string 'PortalTarefas' não configurada. Use
'dotnet user-secrets set ConnectionStrings:PortalTarefas ... …
```

SBOM e análise de vulnerabilidades

```
===== SBOM (SPDX) da imagem com Syft =====
pacotes inventariados: 160
por ecossistema: {'deb': 97, 'nuget': 56, 'outros': 6, 'oci': 1}
  Microsoft.EntityFrameworkCore: ['10.0.14']
  Dapper: ['2.1.89']
  OpenTelemetry: ['1.20.1']
  Microsoft.AspNetCore.Identity.EntityFrameworkCore: ['10.0.14']

$ dotnet list src/PortalTarefas.Web package --vulnerable --include-transitive
  All projects are up-to-date for restore.
The following sources were used:
   https://api.nuget.org/v3/index.json
The given project `PortalTarefas.Web` has no vulnerable packages given the current sources.

$ npm audit --omit=dev --audit-level=high   (front-end)
found 0 vulnerabilities
```

Build a partir de clone novo

```
$ git clone <repositório> && cd PortalTarefas   # clone novo, sem bin/obj/node_modules
ac17bf4 Atividade 8: proxy TLS no compose e pasta gravável de chaves para o usuário não-root

$ dotnet --version && node --version && npm --version
10.0.114
```

```
v22.23.4
10.9.9

$ dotnet build -c Release
Build succeeded.
    0 Warning(s)

$ dotnet test -c Release --no-build
Passed!  - Failed:     0, Passed:    70, Skipped:     0, Total:    70, Duration: 7 s - PortalTarefas.Tests.dll
(net10.0)

$ npm ci && npx vitest run && npm run build
npm ci ok
      Tests  22 passed (22)
✓ built in 161ms

-- git status após o build:
(vazio = nenhum arquivo gerado/alterado pendente)
```